Guía De Laboratorio 09

Sistema Inteligente de Apoyo Pedagógico y Gestión del Aprendizaje para Instituciones Educativas

INTEGRANTES – GRUPO 6:

Rivera Castañeda, Jesús Martín	- U21213646
Quispe Fernández, Alonso	- U20208103
Alanya Guevara, Ben Andre	- U21315933
Ninatanta Fernández, Renato Ventura	- U21205210

ASIGNATURA:

Curso Integrador II: Software

DOCENTE:

Barturen Huaman, Renzo Renato

SECCIÓN:

31738

Lima – Perú

Índice

1. Resultado y alcance del incremento 3
1.1 Producto mínimo de la semana 3
2. Correspondencia con el sílabo 3
3. Fundamentos para cerrar la versión 1 3
3.1 Versión candidata y versión liberada 3
3.2 Cadena de evidencia 4
3.3 Gate de calidad y criterios de salida 4
4. Arquitectura del incremento 4
5. Sesión 17: estabilización técnica 5
5.1 Actividad 1 — Preparar la versión candidata 5
5.2 Actividad 2 — Configurar y ejecutar el gate 5
5.3 Actividad 3 — Verificar acceso a datos y calidad 6
5.4 Actividad 4 — Revisar controles de seguridad 6
5.5 Actividad 5 — Corregir un defecto y repetir la regresión 6
6. Sesión 18: liberación y demostración 7
6.1 Actividad 6 — Construir un paquete inmutable 7
6.2 Actividad 7 — Desplegar la misma versión en cloud 7
6.3 Actividad 8 — Ejecutar la demostración técnica 7
6.4 Actividad 9 — Reunir evidencia verificable 8
6.5 Actividad 10 — Sustentar la versión 1 8
7. Matriz de evidencia APF2 9
7.1 Rúbrica técnica de referencia (20 puntos) 9
8. Riesgos conectados al incremento 10
9. Entregables y estado de evidencia (E0–E11) 10
10. Checklist de cierre 11

1. Resultado y alcance del incremento

Este avance corresponde a la Semana 9 del Curso Integrador II: Software y consolida el trabajo de las Semanas 6, 7 y 8 sobre ESTUD-IA: catálogo y contenidos con MySQL (Semana 6), flujo autenticado con Sanctum para enviar un intento de ejercicio (Semana 7), y pruebas y despliegue con Docker y Caddy (Semana 8). El objetivo es convertir el software probado en una versión candidata verificable, desplegarla en cloud y demostrar que acceso a datos, seguridad, validación y despliegue funcionan juntos. La semana 9 corresponde al APF2 (20 % del promedio final).

Se trabaja sobre la misma aplicación React + Laravel + MySQL. No se crea un proyecto nuevo ni se reemplaza una pieza faltante con diapositivas o respuestas simuladas: si falta algo de las semanas 6 a 8, se regresa a esa actividad.

1.1 Producto mínimo de la semana

- Gate de calidad reproducible (script único) que se detiene cuando falla una comprobación obligatoria.
- Suite de integración separada de la unitaria, con reporte de cobertura de PHPUnit (piso didáctico de 50 %).
- Reporte de calidad de datos en MySQL: duplicados, huérfanos, public_id nulos y eventos de auditoría.
- Revisión pasiva de seguridad con ZAP Baseline sobre la URL HTTPS autorizada, más pruebas 401, 403, 422 y 409.
- Un defecto real corregido con prueba de regresión.
- Paquete inmutable etiquetado por commit (apf2-<hash>), con checksum SHA-256, desplegado sin recompilar en la VM.
- Carpeta de evidencia con manifiesto SHA256SUMS, sin secretos, y sustentación técnica de unos doce minutos.

2. Correspondencia con el sílabo

Sesión	Tema del sílabo	Aplicación en ESTUD-IA
17	Recapitulación de los temas abordados y asesoría sobre el avance.	Diagnóstico técnico completo, corrección del último defecto reproducible y decisión de si la versión candidata puede empaquetarse.
18	Evidencia de acceso a datos, controles de seguridad, validación del servicio y despliegue a producción versión 1.	Construir una versión identificada, desplegarla en cloud y demostrar su comportamiento con evidencias.

El sílabo menciona la retrospectiva del Sprint. Se concreta como una síntesis técnica de cinco líneas: síntoma, causa, archivo modificado, prueba añadida y resultado final del defecto corregido.

3. Fundamentos para cerrar la versión 1

3.1 Versión candidata y versión liberada

Una versión candidata es un conjunto concreto de código, configuración y migraciones que puede evaluarse sin cambios adicionales; se identifica con un commit. La versión liberada agrega los artefactos construidos

desde ese commit y las evidencias de que funcionan en el entorno elegido. La etiqueta apf2-<hash corto> relaciona el despliegue con Git; el SHA-256 del TAR confirma que el paquete transferido no cambió; el ID de imagen relaciona el contenedor en ejecución con la imagen construida. Los tres identificadores responden preguntas distintas y se conservan.

3.2 Cadena de evidencia

Afirmación	Ejecución	Evidencia
El envío de intento evita duplicados	Dos solicitudes concurrentes al mismo ejercicio	Una respuesta 201, una 409 y una sola fila en intento_ejercicio.
La autorización distingue roles	Anónimo, Docente y Alumno sobre POST de intentos	401, 403 y 201 o 409 según la condición.
El esquema está controlado	php artisan migrate durante el arranque y migrate:status	Migraciones aplicadas en orden y sin edición retroactiva.
El despliegue está listo	Compose espera salud y el smoke consulta la URL pública	Servicios healthy y sondas del smoke aprobadas.
La versión es reproducible	Construcción desde commit limpio y paquete SHA-256	Commit, APP_VERSION, IDs de imagen y checksum coincidentes.

3.3 Gate de calidad y criterios de salida

Un gate es una secuencia que se detiene cuando falla una condición obligatoria: PHPUnit verifica el backend, Vitest y el build de producción verifican React, Compose valida y levanta los servicios y el smoke confirma la URL. Playwright y k6 se habilitan cuando están instalados; si no se ejecutan, se registran como pendientes y no como aprobados. La cobertura mide líneas ejecutadas, no corrección: un caso que comprueba 201, 409, auditoría y número de filas demuestra más que uno que solo verifica que la respuesta no sea nula.

Condición	Decisión
Hay una prueba funcional o de integración fallida	Bloquea la versión hasta corregir y repetir la regresión.
Existe doble envío de intento o pérdida de persistencia	Bloquea la versión; compromete la regla central del producto.
Un secreto (APP_KEY, token Sanctum, contraseña) aparece en Git, reportes o capturas	Bloquea la entrega; rotar la credencial y limpiar la evidencia.
HTTPS inválido o la salud no responde	Bloquea la demostración cloud.
ZAP devuelve WARN sin FAIL	Requiere revisión y justificación; no equivale a una vulnerabilidad explotable.
Una herramienta no pudo ejecutarse	Marcar Bloqueado, conservar el diagnóstico y no atribuir un resultado.

4. Arquitectura del incremento

La guía oficial usa Spring Boot, Maven Failsafe, JaCoCo y Testcontainers. En ESTUD-IA se conserva la intención de cada pieza con las herramientas reales del stack Laravel + MySQL + React.

Pieza de la guía (Spring Boot)	Equivalente en ESTUD-IA	Archivo / comando
Failsafe: pruebas IT en la fase verify	Suite de integración separada en phpunit.xml (Unit e Integration), ejecutada con php artisan test --testsuite=Integration	backend/phpunit.xml; tests/Feature/Integration/
Renombrar FunctionalApiTest a FunctionalApiIT	Mover la prueba de la Semana 8 a la suite Integration, sin conservar dos copias	FunctionalApiTest.php en tests/Feature/Integration/
JaCoCo con piso de 50 %	PHPUnit con cobertura (PCOV o Xdebug) y umbral mínimo: php artisan test --coverage --min=50	Reporte en backend/coverage/
Flyway V1 a V5 e historial	Migraciones de Laravel y tabla migrations	php artisan migrate:status
Repository y @Transactional	Service con DB::transaction, lockForUpdate y restricción UNIQUE (Semana 7); Repository/Eloquent para la proyección	IntentoEjercicioService.php
quality-checks.sql en PostgreSQL	Consultas equivalentes en MySQL 8 ejecutadas con el cliente mysql dentro del contenedor	deploy/quality-checks.sql
ZAP Baseline + zap-rules.tsv	Igual, sin cambios: es independiente del framework	deploy/zap-rules.tsv
package-release.sh y collect-apf2-evidence.sh	Mismos scripts con imágenes api (Laravel) y web (React + Caddy)	scripts/

5. Sesión 17: estabilización técnica

La primera sesión usa el entorno local controlado. Al finalizar debe existir un commit candidato, una ejecución repetible del gate, un reporte de calidad de datos, una revisión pasiva de seguridad y una corrección demostrable.

5.1 Actividad 1 — Preparar la versión candidata

- Confirmar que backend, frontend, deploy, tests y scripts pertenecen al mismo repositorio, y registrar git status sin borrar cambios ajenos.
- Mover la prueba de integración de la Semana 8 a tests/Feature/Integration/ y declarar la suite en phpunit.xml; no conservar dos clases equivalentes que reserven el mismo fixture.
- Crear las carpetas scripts y evidence, asignar permisos de ejecución y validar la sintaxis con bash -n.

Evidencia E1: hash del commit inicial, estado del repositorio y validaciones de sintaxis sin errores.

5.2 Actividad 2 — Configurar y ejecutar el gate

- Ejecutar composer install y php artisan test una vez; confirmar que la suite Integration se descubre.

- Generar cobertura con --min=50. Si queda por debajo, identificar clases críticas sin pruebas (por ejemplo AuthService e IntentoEjercicioService) y añadir casos con aserciones útiles antes de tocar el umbral.
- En frontend: npm ci, npm test -- --watch=false y npm run build.
- Validar y levantar Compose, y ejecutar el smoke. Playwright y k6 se incluyen solo si están instalados; de lo contrario se registran como pendientes.

Evidencia E2: reportes de PHPUnit y cobertura, build de React, salida del smoke y estado de Compose.

5.3 Actividad 3 — Verificar acceso a datos y calidad

La explicación debe mostrar código y comportamiento; nombrar el patrón sin señalar dónde se usa no es evidencia.

Elemento	Qué mostrar	Qué demuestra
Service y Repository	IntentoEjercicioService y la consulta que usa (Eloquent o stored procedure de la Semana 6)	La lógica no construye SQL en el Controller ni depende de detalles del motor.
Transacción	DB::transaction que agrupa bloqueo del ejercicio, inserción del intento y evento de auditoría	La operación y sus efectos se confirman o revierten juntos.
Concurrencia	lockForUpdate y la restricción uk_intento_ejercicio	Dos solicitudes no producen dos intentos.
Migraciones	Salida de php artisan migrate:status	El esquema se crea de forma ordenada y verificable.
Calidad de datos	quality-checks.sql: duplicados por (alumno, ejercicio), intentos huérfanos, users sin public_id y conteo de evento_auditoria	La base desplegada mantiene reglas observables.

Las consultas de duplicados, huérfanos y public_id nulos deben devolver cero filas; evento_auditoria debe contener al menos los eventos de los envíos sintéticos. Si un nombre de columna difiere, se adapta solo la consulta y se explica la equivalencia.

Evidencia E3: fragmentos de Service, historial de migraciones, resultado SQL y prueba concurrente con una sola fila.

5.4 Actividad 4 — Revisar controles de seguridad

- Ejecutar las pruebas que rechazan acceso anónimo (401), rol incorrecto (403), entrada inválida (422), token vencido, token alterado y token inexistente en personal_access_tokens.
- Comprobar que las respuestas no contienen contraseñas, APP_KEY, trazas de Laravel ni consultas SQL (APP_DEBUG=false en el despliegue).
- Ejecutar ZAP Baseline solo contra la URL HTTPS autorizada. Interpretar el código de salida: 0 sin WARN ni FAIL, 1 al menos un FAIL, 2 WARN sin FAIL, 3 otro error. ZAP no reemplaza pruebas autenticadas ni un pentest.

Evidencia E4: resultados de autorización y token, reporte ZAP revisado y decisión sobre cada hallazgo.

5.5 Actividad 5 — Corregir un defecto y repetir la regresión

Se selecciona un fallo real y reproducible de las actividades anteriores; no se inventa un defecto ya corregido. Primero se añade una prueba que falle, luego se aplica el cambio mínimo y se ejecutan la prueba específica, el módulo relacionado y el gate completo. Se documenta en cinco líneas:

Campo	Contenido a completar con el defecto real del equipo
Síntoma	Qué se observó y cómo se reprodujo.
Causa	Por qué ocurría.
Cambio	Archivo y cambio mínimo aplicado.
Prueba	Nombre del caso nuevo y su salida inicial fallida.
Resultado	Caso específico y gate completo aprobados en el commit indicado.

Evidencia E5: salida fallida inicial, cambio de código, prueba de regresión y salida aprobada del gate.

6. Sesión 18: liberación y demostración

La segunda sesión comienza solo cuando el gate local está aprobado o los bloqueos están registrados. El despliegue usa las mismas imágenes comprobadas; no se recompila dentro de la VM.

6.1 Actividad 6 — Construir un paquete inmutable

- Confirmar que el repositorio no tiene cambios sin registrar y crear el commit candidato revisando que no incluya .env, tokens ni reportes con secretos.
- Ejecutar el script de empaquetado: obtiene el hash corto, construye las imágenes api y web etiquetadas apf2-<hash>, levanta el entorno, ejecuta el smoke y solo entonces guarda el TAR con docker save.
- Verificar el SHA-256 y revisar version.txt: APP_VERSION, commit e IDs de imagen deben corresponder a la misma ejecución. El checksum se conserva junto al archivo.

Evidencia E6: commit limpio, version.txt, identidades de imagen y checksum aprobado.

6.2 Actividad 7 — Desplegar la misma versión en cloud

Se transfiere el TAR y su checksum a la VM autorizada, se verifica con sha256sum -c, se carga con docker load y se arranca con --no-build y --wait. La VM ya contiene compose.yaml, el .env protegido y los volúmenes de la Semana 8; el .env nunca se copia desde el repositorio. No se ejecuta down con eliminación de volúmenes, no se borra db_data ni se alteran migraciones aplicadas. Si un servicio queda unhealthy, se inspeccionan ps y logs antes de reintentar.

Evidencia E7: checksum en la VM, salida de docker load, servicios healthy e imágenes con la etiqueta candidata.

6.3 Actividad 8 — Ejecutar la demostración técnica

Se ejecutan el smoke, Playwright (lectura y una única escritura sobre un ejercicio libre) y k6 desde una computadora externa a la VM. Un ejercicio ya consumido no se reutiliza para forzar un 201.

Paso	Acción observable	Resultado mínimo
1	Abrir el catálogo y buscar MAT101 en el fixture	Cursos y contenidos provienen de la API real.

Paso	Acción observable	Resultado mínimo
2	Intentar enviar un intento sin token	La API devuelve 401 y no escribe intento_ejercicio.
3	Ingresar como Docente e intentar enviar	La API devuelve 403.
4	Ingresar como Alumno y enviar un ejercicio libre	La API devuelve 201 y la interfaz muestra confirmación.
5	Repetir el mismo ejercicio	La API devuelve 409 y conserva un solo intento.
6	Reiniciar o recrear solamente api	El intento persiste porque MySQL y su volumen continúan.
7	Consultar salud, auditoría y métricas de carga	Salud correcta, evento de auditoría con correlación y resultados k6 medidos.

Evidencia E8: smoke externo, reportes de Playwright y k6, respuesta 201, conflicto 409, control 403 y persistencia tras el reinicio.

6.4 Actividad 9

El colector copia solo rutas conocidas y genera SHA256SUMS. Antes de entregar se revisa cada archivo y se eliminan o redactan tokens, contraseñas, correos reales, cabeceras Authorization y datos personales. No se incluyen el .env, el TAR completo ni logs generales de la aplicación.

Carpeta o archivo	Contenido esperado
git commit y status	Código exacto y estado al recopilar.
versions.txt	Versiones de PHP, Composer, Node, npm, MySQL, Docker y Compose.
tests	Reportes de PHPUnit, Vitest y Playwright.
quality	Cobertura, resultado de quality-checks, ZAP y resumen k6.
deployment	Estado de Compose, imágenes, version.txt, IDs y checksum.
SHA256SUMS	Hash de cada archivo del conjunto de evidencia.

Evidencia E9: carpeta revisada, manifiesto SHA-256 aprobado y ausencia verificada de secretos.

6.5 Actividad 10

La sustentación dura unos doce minutos más tres de preguntas. Una persona puede conducir la navegación, pero cualquier integrante debe poder explicar la capa que implementó. Se muestran código, ejecución y resultados, no una presentación de planificación.

Minuto	Demostración
0 a 1	Identificar commit, APP_VERSION, URL y arquitectura ejecutada (React, Laravel, MySQL, Caddy).
1 a 4	Mostrar Service y transacción, migraciones y calidad de datos.
4 a 7	Ejecutar autenticación, autorización, envío de intento, conflicto y auditoría.

Minuto	Demostración
7 a 9	Mostrar gate, cobertura, Playwright, k6 y ZAP con sus límites explicados.
9 a 11	Comprobar imágenes, salud, HTTPS, persistencia y checksum.
11 a 12	Presentar el defecto corregido y la prueba que evita su regresión.

Preguntas técnicas esperadas: qué evita que dos solicitudes registren el mismo intento (lockForUpdate más UNIQUE); por qué la identidad del alumno sale del token de Sanctum y no del cuerpo; qué diferencia hay entre prueba unitaria, integración con MySQL real y smoke cloud; cómo se demuestra que la imagen desplegada corresponde al commit; qué evidencia existe de que los datos sobreviven a la recreación de la API; y qué no demuestran un reporte de cobertura ni un escaneo ZAP Baseline.

Evidencia E10: demostración ejecutada, respuestas técnicas coherentes y matriz final de requisitos con estado real.

7. Matriz de evidencia APF2

El estado se completa con Aprobado, Falló o Bloqueado, enlazando el archivo concreto. No se escribe Aprobado si la columna de evidencia está vacía. Todo queda en Completar hasta ejecutar.

ID	Condición	Evidencia	Estado
DAT01	Migraciones aplicadas en orden	migrate:status	Completar
DAT02	Envío transaccional sin duplicados	Prueba de integración y SQL	Completar
SEC01	Anónimo y rol incorrecto rechazados	401 y 403	Completar
SEC02	Token inválido y entrada incorrecta rechazados	401 y 422	Completar
FUN01	Consulta y envío funcionan de extremo a extremo	Playwright	Completar
NF01	Carga breve cumple umbrales del laboratorio	Resumen k6	Completar
DEP01	La versión exacta responde por HTTPS	Smoke e imágenes	Completar
DEP02	El intento persiste al recrear api	Consulta antes y después	Completar

7.1 Rúbrica técnica de referencia (20 puntos)

Criterio	Puntos	Desempeño completo
Acceso a datos	4	Service, transacción, migraciones, restricciones y calidad comprobados con código y ejecución.
Seguridad	4	Autenticación, roles, validación, token y revisión pasiva evidenciados sin exponer secretos.
Validación	4	Gate reproducible, pruebas por nivel, cobertura y métricas interpretadas correctamente.

Criterio	Puntos	Desempeño completo
Despliegue	4	Misma versión local y cloud, HTTPS, salud, persistencia, IDs y checksum comprobados.
Explicación técnica	4	Matriz trazable, defecto corregido y respuestas coherentes sobre decisiones y límites.

8. Riesgos conectados al incremento

Se retoman los riesgos R-01 a R-16 de los avances anteriores y se agregan dos propios de esta semana, originados en las herramientas equivalentes del stack PHP.

ID	Riesgo formulado	Disparador	Control / respuesta	Indicador
R-17	Si la imagen de PHP no incluye PCOV o Xdebug, el reporte de cobertura podría salir en cero o no generarse, y el gate no podría verificar el piso de 50 %.	php artisan test --coverage informa que no hay driver de cobertura.	Instalar PCOV en la imagen de pruebas y registrar su versión en versions.txt; no excluir clases para subir el porcentaje.	DEL-01
R-18	Si la carpeta de evidencia incluye .env, APP_KEY, tokens de Sanctum o correos reales, se expondrían credenciales al entregar el APF2.	grep de la carpeta evidence/ encuentra un valor con formato de secreto.	Colector con rutas permitidas, revisión manual previa y rotación inmediata de cualquier credencial expuesta.	SLI-01

9. Entregables y estado de evidencia (E0–E11)

ID	Entregable	Contenido mínimo	Estado
E0	Inicio controlado	Issue, rama feature/semana9-version1 y commit base.	Pendiente
E1	Versión candidata	Hash inicial, estado del repositorio y sintaxis de scripts.	Pendiente
E2	Gate de calidad	PHPUnit, cobertura, build de React, smoke y Compose.	Pendiente
E3	Datos	Service y transacción, migrate:status, resultado SQL y prueba concurrente.	Pendiente
E4	Seguridad	Pruebas 401, 403, 422 y token inválido; reporte ZAP revisado.	Pendiente
E5	Defecto corregido	Falla inicial, cambio, prueba de regresión y gate aprobado.	Pendiente
E6	Paquete inmutable	Commit limpio, version.txt, IDs de imagen y checksum.	Pendiente
E7	Despliegue cloud	Checksum en la VM, docker load y servicios healthy.	Pendiente
E8	Demostración	Smoke externo, Playwright, k6, 201, 409, 403 y persistencia.	Pendiente

ID	Entregable	Contenido mínimo	Estado
E9	Carpeta de evidencia	Manifiesto SHA256SUMS aprobado y ausencia de secretos.	Pendiente
E10	Sustentación	Demostración ejecutada y matriz final con estado real.	Pendiente
E11	Defensa	Presentación y Pull Request con revisión por pares.	Pendiente

10. Checklist de cierre

- El commit presentado coincide con version.txt y con la etiqueta de las dos imágenes.
- php artisan test, las pruebas de React y el build de producción terminan sin fallos.
- La prueba de integración usa MySQL real de pruebas y la demostración usa la base desplegada.
- Las respuestas 401, 403, 422, 201 y 409 se observan donde corresponden.
- Compose informa healthy y la URL externa usa HTTPS válido.
- El intento sigue existiendo después de reiniciar o recrear api.
- La carpeta de evidencia no contiene .env, APP_KEY, tokens, contraseñas ni datos reales.
- Toda prueba omitida aparece como pendiente o bloqueada, nunca como aprobada.
- Los riesgos R-17 y R-18 quedan vinculados al registro de riesgos de ESTUD-IA.